

Rapport technique

Projet 3 — Avis et alertes Montréal (PWA + Backend)

Adam Drabo

1. Qualités et faiblesses du code

L'architecture du projet respecte une séparation claire des responsabilités. Le backend Express est organisé en trois couches distinctes : les routes dans *backend/routes/*, la logique de normalisation dans *backend/utils/normaliser.js*, et la connexion à la base de données dans *backend/db/*. Cette modularité facilite la maintenance et les modifications futures.

La gestion des erreurs est cohérente dans l'ensemble du backend. Chaque route retourne des réponses JSON structurées avec les codes HTTP appropriés — 201 pour la création d'un abonnement, 404 si un abonnement est introuvable, 410 pour les abonnements expirés. Les abonnements expirés sont automatiquement supprimés de MongoDB lors d'un envoi de notification.

Côté frontend, le découplage entre les composants d'affichage et la logique métier est bien respecté. *FiltresAlertes.jsx* ne gère que l'affichage — toute la logique de filtrage est dans *Accueil.jsx*. Le Service Worker dans *public/sw.js* est bien structuré avec des stratégies de cache distinctes pour les assets statiques et les données API.

Une faiblesse notable est la duplication de la constante `BACKEND_URL` dans deux fichiers — *service/alertes.js* et *components/ModaleAbonnement.jsx*. Une variable d'environnement Vite aurait été plus propre et aurait évité les erreurs lors du déploiement.

2. Qualités et faiblesses de l'application

L'application reproduit fidèlement la structure générale et le contenu du site de la Ville de Montréal — titre, liste des alertes, badges de sujets, page de détail. Cependant, l'interface des filtres s'écarte significativement du design original. Le site de la Ville utilise des boutons chip discrets qui ouvrent des modales de sélection, tandis que notre implémentation affiche directement toutes les options sous forme de listes de cases à cocher. Cette approche est fonctionnellement équivalente mais visuellement différente.

Les filtres enrichis avec sélection multiple et chips actifs améliorent l'UX par rapport au projet 2. La bannière hors-ligne informe clairement l'utilisateur quand il n'a pas de connexion.

Les scores Lighthouse en production sont excellents — Performance 100, Accessibility 90, Best Practices 100, SEO 82. Le mode hors-ligne fonctionne correctement grâce à la stratégie

Stale-While-Revalidate pour les données API.

La principale faiblesse de l'application est la fiabilité des notifications push sur desktop. Les notifications fonctionnent parfaitement sur iPhone via Safari mais ne s'affichent pas sur Chrome desktop. Cette limitation est liée aux restrictions des navigateurs desktop concernant les PWA.

3. Propositions d'améliorations

Priorité 1 — Variable d'environnement pour l'URL du backend

Remplacer la constante `BACKEND_URL` codée en dur dans deux fichiers par une variable d'environnement `VITE_BACKEND_URL`. Cela éviterait les erreurs lors du déploiement et centraliserait la configuration en un seul endroit.

Priorité 2 — Mise en cache côté serveur

Ajouter un système de cache côté serveur dans *backend/routes/avis.js* pour stocker temporairement les données de l'API de la Ville. Actuellement, chaque requête du frontend déclenche un appel à l'API externe. Un cache de 5 minutes réduirait la latence et le nombre de requêtes vers l'API de la Ville.

Priorité 3 — Pagination des alertes

Implémenter une pagination dans *Accueil.jsx* pour limiter l'affichage à 10 alertes par page avec un bouton Charger plus. Actuellement, les 100 alertes sont chargées et affichées en une seule fois, ce qui peut ralentir le rendu sur les appareils moins puissants.